Diario Prácticas

Lucía Santos Muñoz

Contents

Detalles del Proyecto . . . 5

1 Week: 14 to 18 of september . . . 7

2 Wedenesday 16/09 . . . 9

3 Thursday 17/09 and Friday 18/09 . . . 11
3.1 Readind Cata Thesis . . . 11
3.2 First analysis with SpaceRanger . . . 11
3.3 Search Segmentation Tools . . . 11

4 Week: 21 to 25 of september . . . 13

5 Monday 22/09 . . . 15
5.1 Investigate other tools . . . 15
5.2 SMURF . . . 15
5.3 Installation of SpaceRanger . . . 18

6 Tuesday 23/09 . . . 19
6.1 First analysis with SpaceRanger . . . 19
6.2 SMURF . . . 20

7 Wednesday 23/09 . . . 21

8 Thursday 24/09 . . . 23
8.1 SR solution . . . 23
8.2 TMA_7_8 . . . 23
8.3 Cellpose . . . 23

9 Friday 25/09/2026 . . . 25
9.1 SR . . . 25
9.2 SMURF . . . 26
9.3 Cellpose . . . 26

10 Semana del 21 al 25 de Septiembre . . . 27

11 Lunes 21/09 . . . 29
11.1 Script T3_3_Pc.py . . . 29
11.2 Script T3_0_clusters . . . 29

12 FUTURO . . . 31

13 PENDIENTE A FUTURO 33

Detalles del Proyecto

- **Título:** Desarrollo de un pipeline bioinformático para la clasificación de integrones de clase 1 por especie y ambiente.
- **Autora:** Lucía Santos Muñoz

1. Week: 14 to 18 of september

2. Wedenesday 16/09

- CNIO mail created
- Cluster usage request sent

Note: Important data to look at in the web report from SpaceRanger is **Sequencing Saturation**. Its value should be 80-90% to consider good quality data. Also, **fraction of bins under tissue 8um**, should be 80-90%, not decreasing too much after segmentation process.

3. Thursday 17/09 and Friday 18/09

3.1 Readind Cata Thesis

The most highlatable information extracted from the thesis is the following:

- **BLCA cells**: Epithelial-to-mesenchymal transition (EMT) losing urothelial diferentiation. Improve invasion.
- **SM cells**: myCAF features.

*Upregulation of extracellular matrix (ECM) genes –> collagens
[Molecular + Mechanotransduction] pathways –> Integrated signaling network –> cells adapt to changes in tissue mechanics -[dysregulation]–> Pathological remodeling an Disease progression
Normal urothelium –> Init (Common process) –> * Papillary Tumor (NI) - FGFR3 * Non-Papillary Tumor (I)

3.2 First analysis with SpaceRanger

First started with 3_4 sample, because it is the sample used by the Pathologist to annotate the tissue. We run SpaceRanger in the web-interface, following the next steps: 1. Select library –> Visium 2. Inputs: * FASTQ files * Tissue image (H&E) * CytAssist image 3. Create New Analysis 4. Advanced Options: * Reference: human GRCh38-2024-A * Probe Set: human 2024 * json LoupeBrowser (alignment cytassist+H&E) 5. Run Analysis
Everyting was run with the _clean image.
Results 1: Good in general, but the total UMIs in cells is very low, 44,3%. We have to increase the number, at least 80%.
Then, we repeated the analysis but this time modifying the parameter max.diameter instead of 256, 1024 pixels. And a third time with the cut image (only the good sample).
Results 2 and 3: Exactly the same UMIs %.

3.3 Search Segmentation Tools

Searched other option for segmentation.

- CellVit++
- Bin2Cell == SapceRanger (StarDist)
- Cellpose –> Jupyter Notebook Drive
- SMURF –> Deep Learning (2nd step soft segmentation; increase UMI counts)

4. Week: 21 to 25 of september

5. Monday 22/09

5.1 Investigate other tools

I investigated other options to perform the analysis of the samples. First I read about StarDist, that use image-mask pairs cropped from the original image into different regions (128x128) to train a model (split into test and train). Also it raises the option of doing data augmentation by geometrically distorting the input images.

Also, I read about SMURF and started writing a script for when we finish with SpaceRanger, beacuse the SR outputs are the SMURF inputs.

5.2 SMURF

I installed SMURF the week before with an srun command, following the instructions in the SMURF manual, but as mentioned before I couldn't start running any analysis due to the inputs required. So, I started writing the python script, *test_smurf.py*:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import scanpy as sc
import anndata

import smurf as su

import pickle
import copy
import gzip

import os

data_path = 'smurf_tma_3_4_data/'
save_path = 'results_tm_3_4/'

# Check if the directory exists

if not os.path.exists(save_path):
    os.makedirs(save_path)

# First step to build so (spatial object)

so_3_4  =  su.prepare_dataframe_image(data_path+'binned_outputs/square_002um/spatial/
tissue_positions.parquet',
                            data_path+'AL-7332_99B-15848-3D_3_97-47941S_4_clean.tif',
#change file name depending on the sample
                                    'HE')

plt.imshow(so_3_4.image_temp())
plt.show()
plt.close()

# Please use your own tools to perform cell segmentation on so.image_temp()
# SpaceRanger for .image_temp() ???   --> More simple since you don't have to edit
```

```
any image
# Another option is to cut the SpaceRanger segmentation output

#SMURF tutorial, maybe executed the segmentation with StarDist, generating a .npy
#? I think it is possible to generate a .npy from a .tiff (nucleus_instance_mask.tiff
from SpaceRanger)

# In this web (https://www.kaggle.com/code/adudp007/loading-tiff-image-to-numpy-array)
# they load a tiff image to numpy array:

import tifffile

tiff_image = tiff.imread('smurf_segmentation/outs/nucleus_instance_mask.tiff')
array_image = np.array(tiff_image)
# As I'm not sure if it works, first see how the array_image looks like
so_3_4.segmentation_final = array_image

#####

# !!! SEARCH AN ALTERNATIVE, using StarDist or converting .tiff

#Assuming the SR option:

# with gzip.GzipFile(data_path + 'image_to_segmentation.npy.gz', 'w') as f:
#    np.save(f, so.image_temp())
# so.segmentation_final = np.load(data_path+'segmentation_final.npy'



# we start generating cell and spot information to prepare for the analysis.

so_3_4.generate_cell_spots_information()

su.plot_results(so_3_4.image_temp(), so_3_4.segmentation_final ) #check both aligns


with open(save_path + 'so_3_4.pkl', 'rb') as file:
    so_3_4 = pickle.load(file)


# load adata from original 2um results

adata        =        sc.read_10x_mtx(data_path        +'binned_outputs/square_002um/
filtered_feature_bc_matrix')
adata = copy.deepcopy(adata[so_3_4.df[so_3_4.df.in_tissue == 1]['barcode']])
adata.obs

sc.pp.filter_genes(adata, min_counts=1000)


# Generate nuclei*genes matrix

su.nuclei_rna(adata,so_3_4)
adata_sc = copy.deepcopy(so_3_4.final_nuclei)
adata_sc

# Filter cells by counts while preserving the raw data version.

sc.pp.filter_cells(adata_sc, min_counts=5) #Change min_counts??
adata_raw = copy.deepcopy(adata_sc)
```

```
adata_raw

adata_raw = copy.deepcopy(adata_sc)

# Clustering

adata_sc = su.singlecellanalysis(adata_sc,resolution=2)

# Note: Resolution lower values generally merge clusters,
#       whereas higher values tend to split clusters
# Modify resolution in order to the results

su.itering_arragement(adata_sc, adata_raw, adata, so, resolution=2, save_folder =
save_path, show = True, keep_previous = False)

# Results of iteration

adatas_final = sc.read_h5ad(save_path +'adatas.h5ad')

with open(save_path +'cells_final.pkl', 'rb') as file:
    cells_final = pickle.load(file)

with open(save_path +'weights_record.pkl', 'rb') as file:
    weights_record = pickle.load(file)


# save files:
import pickle
import gzip

def save_compressed_pickle(data, filename):
    with gzip.GzipFile(filename, 'wb') as f:
        pickle.dump(data, f, protocol=pickle.HIGHEST_PROTOCOL)


save_compressed_pickle(spots_X_dic, save_path + "spots_X_dic.pkl.gz")
save_compressed_pickle(celltypes_dic, save_path + "celltypes_dic.pkl.gz")
save_compressed_pickle(nonzero_indices_dic, save_path + "nonzero_indices_dic.pkl.gz")
save_compressed_pickle(nonzero_indices_toml,                                save_path
+ "nonzero_indices_toml.pkl.gz")
save_compressed_pickle(cells_X_plus_dic, save_path + "cells_X_plus_dic.pkl.gz")
save_compressed_pickle(cells_before_ml, save_path + "cells_before_ml.pkl.gz")
save_compressed_pickle(cells_before_ml_x, save_path + "cells_before_ml_x.pkl.gz")
save_compressed_pickle(groups_combined, save_path + "groups_combined.pkl.gz")
save_compressed_pickle(spots_id_dic, save_path + "spots_id_dic.pkl.gz")
save_compressed_pickle(spots_id_dic_prop, save_path + "spots_id_dic_prop.pkl.gz")
save_compressed_pickle(pct_toml_dic, save_path + 'pct_toml_dic.pkl.gz')


#load files:
# Define a function to load compressed pickles
def load_compressed_pickle(filename):
    with gzip.GzipFile(filename, 'rb') as f:
        return pickle.load(f)

weight_to_celltype = load_compressed_pickle(save_path + 'weight_to_celltype.pkl.gz')
cells_before_ml = load_compressed_pickle(save_path + 'cells_before_ml.pkl.gz')
groups_combined = load_compressed_pickle(save_path + 'groups_combined.pkl.gz')
pct_toml_dic = load_compressed_pickle(save_path + 'pct_toml_dic.pkl.gz')
nonzero_indices_dic = load_compressed_pickle(save_path + 'nonzero_indices_dic.pkl.gz')
```

```
nonzero_indices_toml          =          load_compressed_pickle(save_path          +
'nonzero_indices_toml.pkl.gz')


# Note: Adjust maximum_cells = 6000 according to your available GPU memory.
# Start conservatively and increase it only after confirming that the preparation step
fits in memory.

pct_toml_dic,  spots_X_dic,  celltypes_dic,  cells_X_plus_dic,  nonzero_indices_dic,
nonzero_indices_toml,     cells_before_ml,     cells_before_ml_x,     groups_combined,
spots_id_dic,spots_id_dic_prop = su.make_preparation(cells_final,
                                                                                so,
adatas_final, adata, weights_record, maximum_cells = 6000)

# Check if SMURF-full workflow is possible

import torch
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(device)

if device.type != "cuda":
    print("Warning: CUDA was not detected. The SMURF-full workflow may be impractically
slow on CPU. Use the SMURF-lite tutorial instead if a GPU is not available.")
```

5.3 Installation of SpaceRanger

I installed SR in the cluster, following the instructions.
/storage/scratch01/users/lsantos/SpaceRanger_TMA/spaceranger-4.1.0/

6. Tuesday 23/09

6.1 First analysis with SpaceRanger

First, before running any analysis in SpaceRanger, I checked the verified the installation of the software following the instructions in the SpaceRanger manual.

Then every file was collected in the cluster, concretly in the folder of the project BLCA_Spatial.

Finally I started the analysis following the instructions in the Visium HD SpaceRanger Manueal.

Also looked file formats and commands.

srun –nodes=1 –ntasks-per-node=1 –mem=15G –time=02:00:00 –pty bash -i

rsync -avx /home/luciasm/CNIO/TMA_3_4/ lsantos@cluster1.cnio.es:/storage/scratch01/groups/ce/BLCA_Spatial/TMA_3_4/

```
sbatch -o log_TMA_3_4_test1.txt -e error_TMA_3_4_test1.txt -
J TMA_3_4_SR_test1 -c 2 --mem=50G -t1440 --wrap="/storage/
scratch01/users/lsantos/SpaceRanger_TMA/spaceranger-4.1.0/spaceranger count --
id=TMA_3_4_test1 --transcriptome=/storage/scratch01/users/lsantos/SpaceRanger_TMA/
refdata-gex-GRCh38-2024-A --fastqs=/storage/scratch01/groups/ce/BLCA_Spatial/
TMA_3_4/fastq_3_4 --probe-set=/storage/scratch01/users/lsantos/SpaceRanger_TMA/
Visium_Human_Transcriptome_Probe_Set_v2.1.0_GRCh38-2024-A.csv --cytaimage=/storage/
scratch01/groups/ce/BLCA_Spatial/TMA_3_4/cytassist_image.tiff --image=/storage/
scratch01/
groups/ce/BLCA_Spatial/TMA_3_4/AL-7332_99B-15848-3D_3_97-47941S_4_clean.tif --loupe-
alignment=/storage/scratch01/groups/ce/BLCA_Spatial/TMA_3_4/H1-C692N9D-D1-fiducials-
image-registration.json --create-bam=false --custom-segmentation-file=/storage/
scratch01/groups/ce/BLCA_Spatial/TMA_3_4/cells.geojson --nucleus-expansion-distance-
micron=8"
```

[error] Invalid –custom-segmentation-file GeoJson provided. Expecting FeatureCollection objects. Error: invalid type: map, expected a string at line 1 column 2

The geojson used was created with CellVit++ so it is not compatible with the format requested, “FeatureCollection”. To convert the geojson to the correct format, the steps are open the file in QuPath and select the object (nucleus detection) and export as a geojson objet, indicating the format “FeatureCollection”.

```
sbatch -o log_TMA_3_4_test1.txt -e error_TMA_3_4_test1.txt -
J TMA_3_4_SR_test1 -c 2 --mem=50G -t1440 --wrap="/storage/
scratch01/users/lsantos/SpaceRanger_TMA/spaceranger-4.1.0/spaceranger count --
id=TMA_3_4_test1 --transcriptome=/storage/scratch01/users/lsantos/SpaceRanger_TMA/
refdata-gex-GRCh38-2024-A --fastqs=/storage/scratch01/groups/ce/BLCA_Spatial/
TMA_3_4/fastq_3_4 --probe-set=/storage/scratch01/users/lsantos/SpaceRanger_TMA/
Visium_Human_Transcriptome_Probe_Set_v2.1.0_GRCh38-2024-A.csv --cytaimage=/storage/
scratch01/groups/
ce/BLCA_Spatial/TMA_3_4/cytassist_image.tiff --image=/storage/scratch01/groups/ce/
BLCA_Spatial/TMA_3_4/AL-7332_99B-15848-3D_3_97-47941S_4_clean.tif --loupe-alignment=/
storage/scratch01/groups/ce/BLCA_Spatial/TMA_3_4/H1-C692N9D-D1-fiducials-image-
registration.json --create-bam=false --custom-segmentation-file=/storage/scratch01/
groups/ce/BLCA_Spatial/TMA_3_4/AL-7332_99B-15848-3D_3_97-47941S_4_clean.geojson --
nucleus-expansion-distance-micron=8"
```

To check if the job submitted correctly, you can use the command:

```
squeue -u lsantos
```

If the job stop suddenly you can check what happened with the commands:

```
cat log_TMA_3_4_test1.txt
cat error_TMA_3_4_test1.txt
```

6.2 SMURF

One of the doubts that I had about SMURF is the third input explained in the SMURF manual, 'segmentation_final.npy'. As indicated it is generated by performing your own segmentation over so.image_temp(), resulting in a NumPy array with the shape as so.image_temp(). – *I think the .npy file comes from StarDist segmentation –> I'm not sure of this beacuse in the documentation they don't mention explicitly the outputs, although they mention:* "The image annotations (also known as label images or label masks) should be integer-valued (e.g. 8-bit, 16-bit, 32-bit) TIFF files where all background pixels have value 0 and each object instance is represented by an area/volume filled with a unique positive integer value. It does not matter what the values are and they do not need to be consecutive. Please note that a foreground/background segmentation mask, where all object instances are denoted by the same value, is not sufficient for StarDist training." *I think this is basicly what SMURF needs.*

SpaceRanger does not generate any NumPy array, the only file that could supply this input is the nucleus_instance_mask.tiff. So, I searched how to convert a tiff file to a numpy array and I found the following code:

```
import tifffile

tiff_image = tiff.imread('smurf_segmentation/outs/nucleus_instance_mask.tiff')
array_image = np.array(tiff_image)
```

Now I have to check if effectively the .tiff file generated by SpaceRanger is compatible with the input of SMURF.

7. Wednesday 23/09

!! PONER AQUÍ UN CUADRO DE ERROR [Error log at:
TMA_3_4_test1/SPATIAL_RNA_COUNTER_CS/SPATIAL_RNA_COUNTER/
SPATIAL_POST_MATRIX_COMP/BIN_HD_DATA/_BIN_HD_DATA/PROCESS_SEG/_PROCESS_SEG/
SEGMENT_TO_FEATURE_CELL_MATRIX/PREPROCESS_NUCLEUS_SEGMENTATION_GEOJSON/
fork0/chnk00-u2aa9b2db95/_errors

Log message:

stage failed unexpectedly: ‘called `Result::unwrap()` on an `Err` value: ExactlyOneError { first: POLYGON((3321.0….]

The previous job failed due to the format of the –custom-segmentation-file, instead of single polygons (individual nucleus), there were multypoligon. The reason of this multi- is because of the 5 different annotations provided in the cells.geojson interpreted as 5 objects with multiple polygons conforming it, interpreting SR as if there were 5 big nucleus. To solve this problem we first looked at the cellsdetection.geojson but the objects were interpreted as multipoints lacking of geometry. We tried also splitting the annotations into single objects, being each nucleus detected as a different object, but the object type changed to “point”. We still needed polygons to make SR work, so cells.geojson were processed in the same way we did with cellsdetection.geojson, splitting the annotations objects into individual ones, obtaining this time the “polygons” object type.

```
sbatch      -o      log_TMA_3_4_test2.txt      -e      error_TMA_3_4_test2.txt      -
J      TMA_3_4_SR_test2      -c      2      --mem=50G      -t1440      --wrap="/storage/
scratch01/users/lsantos/SpaceRanger_TMA/spaceranger-4.1.0/spaceranger     count     --
id=TMA_3_4_test2     --transcriptome=/storage/scratch01/users/lsantos/SpaceRanger_TMA/
refdata-gex-GRCh38-2024-A         --fastqs=/storage/scratch01/groups/ce/BLCA_Spatial/
TMA_3_4/fastq_3_4       --probe-set=/storage/scratch01/users/lsantos/SpaceRanger_TMA/
Visium_Human_Transcriptome_Probe_Set_v2.1.0_GRCh38-2024-A.csv    --cytaimage=/storage/
scratch01/groups/ce/BLCA_Spatial/TMA_3_4/cytassist_image.tiff       --image=/storage/
scratch01/
groups/ce/BLCA_Spatial/TMA_3_4/AL-7332_99B-15848-3D_3_97-47941S_4_clean.tif  --loupe-
alignment=/storage/scratch01/groups/ce/BLCA_Spatial/TMA_3_4/H1-C692N9D-D1-fiducials-
image-registration.json     --create-bam=false     --custom-segmentation-file=/storage/
scratch01/groups/ce/
BLCA_Spatial/TMA_3_4/AL-7332_99B-15848-3D_3_97-47941S_4_clean_cells_split.geojson  --
nucleus-expansion-distance-micron=8"
```

8. Thursday 24/09

8.1 SR solution

The job launched the previous day failed again. In this case the error was because of a polygon with empty coordinates "[]", so to solve this the entire line was deleted. First, I determined the Featue Collection geojson structure, to establish where each object description start: {"type":"Feature","id":"...",...}. Then I manually deleted the corresponding line with a text editor and launched again the job executing the same command but changing the path to the new edited geojson.

```
sbatch -o log_TMA_3_4_test2.txt -e error_TMA_3_4_test2.txt -
J TMA_3_4_SR_test2 -c 2 --mem=50G -t1440 --wrap="/storage/
scratch01/users/lsantos/SpaceRanger_TMA/spaceranger-4.1.0/spaceranger count --
id=TMA_3_4_test2 --transcriptome=/storage/scratch01/users/lsantos/SpaceRanger_TMA/
refdata-gex-GRCh38-2024-A --fastqs=/storage/scratch01/groups/ce/BLCA_Spatial/
TMA_3_4/fastq_3_4 --probe-set=/storage/scratch01/users/lsantos/SpaceRanger_TMA/
Visium_Human_Transcriptome_Probe_Set_v2.1.0_GRCh38-2024-A.csv --cytaimage=/storage/
scratch01/groups/ce/BLCA_Spatial/TMA_3_4/cytassist_image.tiff --image=/storage/
scratch01/
groups/ce/BLCA_Spatial/TMA_3_4/AL-7332_99B-15848-3D_3_97-47941S_4_clean.tif --loupe-
alignment=/storage/scratch01/groups/ce/BLCA_Spatial/TMA_3_4/H1-C692N9D-D1-fiducials-
image-registration.json --create-bam=false --custom-segmentation-file=/storage/
scratch01/groups/ce/BLCA_Spatial/TMA_3_4/
AL-7332_99B-15848-3D_3_97-47941S_4_clean_cells_split_noempty.geojson --nucleus-
expansion-distance-micron=8"
```

8.2 TMA_7_8

Ele sent us an excel with the info. about the quality of each sample (saturation, presence of tumor, histology, etc.), and taking it into account we decided to start analyzing the samples 7_8. They were both classified as "bien", with high seq_saturation and "OK" for hystology-annotation but low UMIs (we will see how this affect to the results).

Then the same process as we did with the sample 3_4 was followed this time (aligning cytassist+image, online SR..) but first I convert the original image into a pyramidal one, to make it compatible with SR (ome.tif).

After 2h we obtained the results and summing up they are worse than what we obtained with samples 3 and 4. The total UMIs in cells are about a 12%. Also, we noticed that the quality of the image decreased a lot in comparison with what we see when we load the image (ome.tif) in QuPath. To check this I would try to see if just the image with the segmentation is an output of SR, to confirm if this lower quality is only because of the report .html format. If the quality is bad in both images, the individual and the added to the report, then we would have to study if using SR in the cloud is not obtaining good results because of the way it process the image.

8.3 Cellpose

I tried Cellpose with a jupyter notebook but couldn't finish the execution because I run out of RAM, so my idea now is to run it in a conda environment. I created the conda env, tomorrow I'll install Cellpose and run the script I wrote named Run_Cellpose.py localized in the ubuntu CNIO folder. *Also, I will try again the jupyter notebook tomorrow.

```
conda activate cellpose
python -m pip install cellpose[gui] #gui is the interactive dependence
#if gui is not possible then run:
```

```
#python -m pip install cellpose

python -m cellpose #start the gui
```

9. Friday 25/09/2026

9.1 SR

Again failed SR, with the same error as yesterday. I reviewed the geojson and there are still Multipolygons. I think the problem is how CellVit++ draw the polygons, so I would try again SR but now using their own nucleus segmentation geojson, just to complete a job and study the output.

```
sbatch -o log_TMA_3_4_SRtest.txt -e error_TMA_3_4_SRtest.txt -
J TMA_3_4_SR_test -c 2 --mem=50G -t1440 --wrap="/storage/
scratch01/users/lsantos/SpaceRanger_TMA/spaceranger-4.1.0/spaceranger count --
id=TMA_3_4_SRtest --transcriptome=/storage/scratch01/users/lsantos/SpaceRanger_TMA/
refdata-gex-GRCh38-2024-A --fastqs=/storage/scratch01/groups/ce/BLCA_Spatial/
TMA_3_4/fastq_3_4 --probe-set=/storage/scratch01/users/lsantos/SpaceRanger_TMA/
Visium_Human_Transcriptome_Probe_Set_v2.1.0_GRCh38-2024-A.csv --cytaimage=/storage/
scratch01/groups/ce/BLCA_Spatial/TMA_3_4/cytassist_image.tiff --image=/storage/
scratch01/
groups/ce/BLCA_Spatial/TMA_3_4/AL-7332_99B-15848-3D_3_97-47941S_4_clean.tif --loupe-
alignment=/storage/scratch01/groups/ce/BLCA_Spatial/TMA_3_4/H1-C692N9D-D1-fiducials-
image-registration.json --create-bam=false --custom-segmentation-file=/storage/
scratch01/groups/ce/BLCA_Spatial/TMA_3_4/nucleus_segmentations.geojson --nucleus-
expansion-distance-micron=8"
```

Also, I launched another SR job but this time without custom segmentation, just to study the output and be able to star with SMURF.

```
sbatch -o log_TMA_3_4_nocustom.txt -e error_TMA_3_4_nocustom.txt -
J TMA_3_4_nocustom -c 2 --mem=50G -t1440 --wrap="/storage/
scratch01/users/lsantos/SpaceRanger_TMA/spaceranger-4.1.0/spaceranger count --
id=TMA_3_4_nocustom --transcriptome=/storage/scratch01/users/lsantos/SpaceRanger_TMA/
refdata-gex-GRCh38-2024-A --fastqs=/storage/scratch01/groups/ce/BLCA_Spatial/
TMA_3_4/fastq_3_4 --probe-set=/storage/scratch01/users/lsantos/SpaceRanger_TMA/
Visium_Human_Transcriptome_Probe_Set_v2.1.0_GRCh38-2024-A.csv --cytaimage=/storage/
scratch01/groups/ce/BLCA_Spatial/TMA_3_4/cytassist_image.tiff --image=/storage/
scratch01/
groups/ce/BLCA_Spatial/TMA_3_4/AL-7332_99B-15848-3D_3_97-47941S_4_clean.tif --loupe-
alignment=/storage/scratch01/groups/ce/BLCA_Spatial/TMA_3_4/H1-C692N9D-D1-fiducials-
image-registration.json --create-bam=false"
```

As the problem is with the geojson, I looked for a tool that help me validating the structure, content, etc. I used it but I realized that it was for geographics coordinates, so didn't helped at all.

```


import geojson-validator
import json

geojson_path = "/home/luciasm/CNIO/
TMA_3_4/AL-7332_99B-15848-3D_3_97-47941S_4_clean_cells_split_noempty.geojson"
with open(geojson_path, 'r') as f:
    geojson_input = json.load(f)

geojson
```

conda create –name geojsonv python=3.12

Finally, Jaime edited the last geojson, the one without [] and divided the MultiPolygons into single polygons. Runned SR again with this new geojson.

```
sbatch -o log_TMA_3_4_Jgj.txt -e error_TMA_3_4_Jgj.txt -J TMA_3_4_Jgj -c 2 --mem=50G
-t1440    --wrap="/storage/scratch01/users/lsantos/SpaceRanger_TMA/spaceranger-4.1.0/
spaceranger    count    --id=TMA_3_4_Jgj    --transcriptome=/storage/scratch01/users/
lsantos/SpaceRanger_TMA/refdata-gex-GRCh38-2024-A --fastqs=/storage/scratch01/groups/
ce/BLCA_Spatial/TMA_3_4/fastq_3_4       --probe-set=/storage/scratch01/users/lsantos/
SpaceRanger_TMA/Visium_Human_Transcriptome_Probe_Set_v2.1.0_GRCh38-2024-A.csv      --
cytaimage=/storage/scratch01/groups/ce/BLCA_Spatial/TMA_3_4/cytassist_image.tiff   --
image=/storage/scratch01/
groups/ce/BLCA_Spatial/TMA_3_4/AL-7332_99B-15848-3D_3_97-47941S_4_clean.tif  --loupe-
alignment=/storage/scratch01/groups/ce/BLCA_Spatial/TMA_3_4/H1-C692N9D-D1-fiducials-
image-registration.json    --create-bam=false    --custom-segmentation-file=/storage/
scratch01/groups/ce/BLCA_Spatial/TMA_3_4/
AL-7332_99B-15848-3D_3_97-47941S_4_clean_cells_split_noempty_SpaceRanger.geojson   --
nucleus-expansion-distance-micron=8"
```

JOB IDs: * TMA_3_4_Jgj {geojson editado, sin MultiPolygons}
* TMA_3_4_nocustom {no se ha usado –custom-segmentation}
* TMA_3_4_SRtest {geojson generado desde la nube de SpaceRanger}

9.2 SMURF

“The segmentation results should be saved as a 2D NumPy array with the same dimensions as the input image. In this array, regions without nuclei should be labeled as `0`, while nucleated regions should be assigned `unique cell IDs`.”

9.3 Cellpose

No gui, so I installed the normal version of cellpose.

```
srun  -p  gpu  --nodes=1  --ntasks-per-node=1  -c  4  --mem=15G  --time=02:00:00  --
gres=gpu:A100:1 --pty bash -i
```

I think I will have to run Run_Cellpose.py (/storage/scratch01/users/lsantos/Cellpose_TMA) because there are no resources to run the srun with gpu

10. Semana del 21 al 25 de Septiembre

11. Lunes 21/09

OBJETIVOS INICIALES:

- Dejar bien hecho scripts T3_3
- Aprender a paralelizar

11.1 Script T3_3_Pc.py

he dejado bien hecho (comentado + traducido) el script de extracción, búsqueda y clasificación de Pcs
FIXME: Aun así, Creo que da problemas porque detecta dos Pc como en la misma región¿ revisar eso -> igual con clustering se solcuionarlo

11.2 Script T3_0_clusters

Sigo sin poder hacer clustering ya que no me da el ordenador, probar ya con la workstation

12. FUTURO

13. PENDIENTE A FUTURO

Jose:

- Preguntar a Jose si hay que trabajar con COMPLETOS o INCOMPLETOS (clasificación que da IF2) -> Modesto opina que mejor completos!!!!
- A qué nivel de assembly_level vamos a trabajar??? -> En función del nivel podríamos definir un –topology-file para IF2 (o directamente –circular)
- Definir las métricas de calidad que vamos a usar para el filtrado!
- Igual sí que podemos quitar los contigs o filtrar por el N50 -> (o directamente hacerlo a la inversa y luego con los datos ya descargados de IF2 filtrar a posterior)

datasets permite descargar también todos los metadatos del genoma (creo recordad que es con la función summary, pero no estoy seguro) y eso te puede venir bien también para clasificar los integrones por el ambiente, hospedador, etc. Es algo que está en el título de la propuesta de Prácticas pero se me había pasado y puede merecer la pena que lo tengas en cuenta ahora.

añadir módulo que genere un TOPOLOGY_FILE para IntegronFinder2 de forma correcta en función del nivel de ensamblado en la T2

Se puede incorporar un umbral de N50 para decidir si hacer el blast en: todo el assembly o en 1 contig concreto –> en función también del assembly_level que hayamos puesto

file es una variable que python rellena automáticamente con la ruta del .py qye se está ejecutando, luego con eso se podría automatizar un poco más algo?????

- Añadido en a futuro, pero poder hacer un TIPADO DE GENOMAS, es decir, buscar la relación y poder clasificar el integrón por el tipo de genoma (ej. A,B,C,D en Ecoli // multilocus sequence: mls) –> ya hay programas que lo hacen (para relacionarlo con la literatura).
  - Esto se hace en entorno conda diferente ya que pueden estar desactualizados y puede dar problemas (coli phylogroups github // pyMLST github)
  - Así conseguimos evitar que la seleccción de genomas (ej. filtrado) sesgue los resultados que obtenemos
- mirar IF2 que detecta integrones de clase 1, 2 y 3 -> ver si los Pc de clase 2 y clase 3 se parecen a los de clase 1 y sino con el blsat se van

se podría acotar aun más la secuencia extraida para reducir carga computacional a, en lugar del integrón entero, la integrasa (que es donde se encuentra el Pc)

posibilidad a añadir que busque en los CALIN en la parte upstream la integradsa (el inciio por ejemplo, 40 pb) y en aquellos qeu dé positivo buscar el Pc (por ejemplo) * las enterobact los integrones que tienen son todos moviles (exógenos), pero bact como vibrio por ejemplo pueden tener calin que se haya fragmetado la Integrasaf